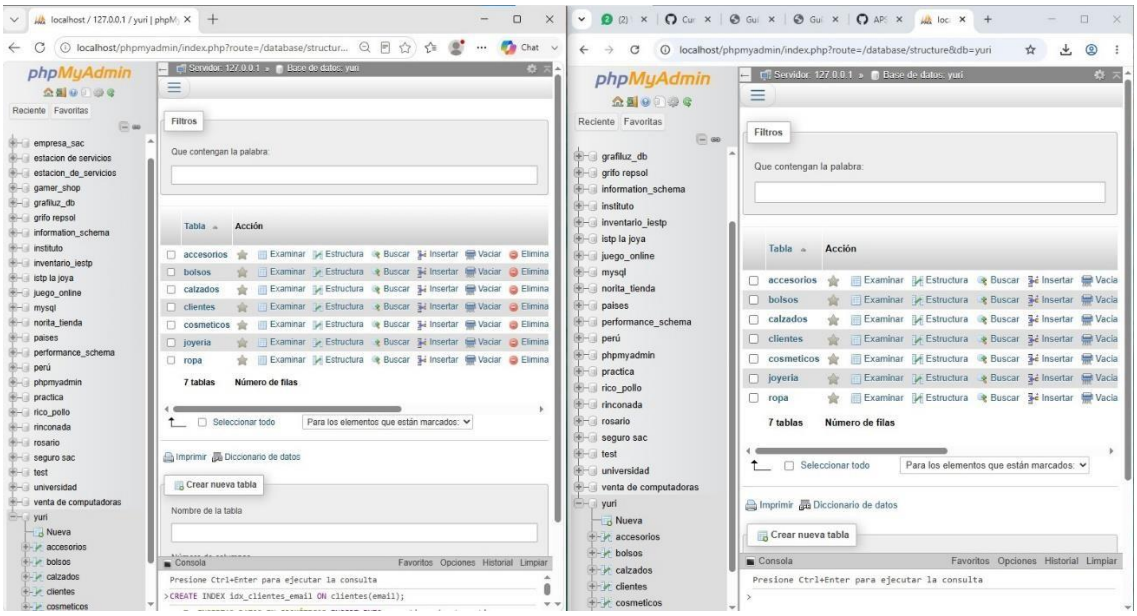
SESION 03: TRAFICO MASIVO Y CONCURRENCIA

Primero en la sesión nos pide dos pestanas del navegador abiertas al mismo tiempo:



En ambas se seleccione la base de datos que es Yuri mi tienda.

ACTIVIDAD NRO1:

Paso 1: agregamos dos columnas nuevas a ítems : es_unico y vendido **código**:

-- Sesión 3: Cambios de estructura sin interrumpir el servicio ("evento de lanzamiento")

-- Motor: MySQL 8.0.29+ / MariaDB 10.0.2+ (soportan ADD COLUMN IF NOT EXISTS) USE

yuri;

-- Marca si un item es único (solo puede existir 1 en todo el juego, ej: skin legendaria de evento)

ALTER TABLE ropa

ADD COLUMN IF NOT EXISTS es_unico BOOLEAN NOT NULL DEFAULT FALSE;

-- Marca si el item único ya fue vendido (evita doble venta)

ALTER TABLE ropa

ADD COLUMN IF NOT EXISTS vendido BOOLEAN NOT NULL DEFAULT FALSE;

-- Nota: en InnoDB, agregar una columna con DEFAULT es una operación "instant" (ALGORITHM=INSTANT)

-- desde MySQL 8.0.12+/MariaDB 10.3+, por lo que no bloquea la tabla ni la reescribe completa

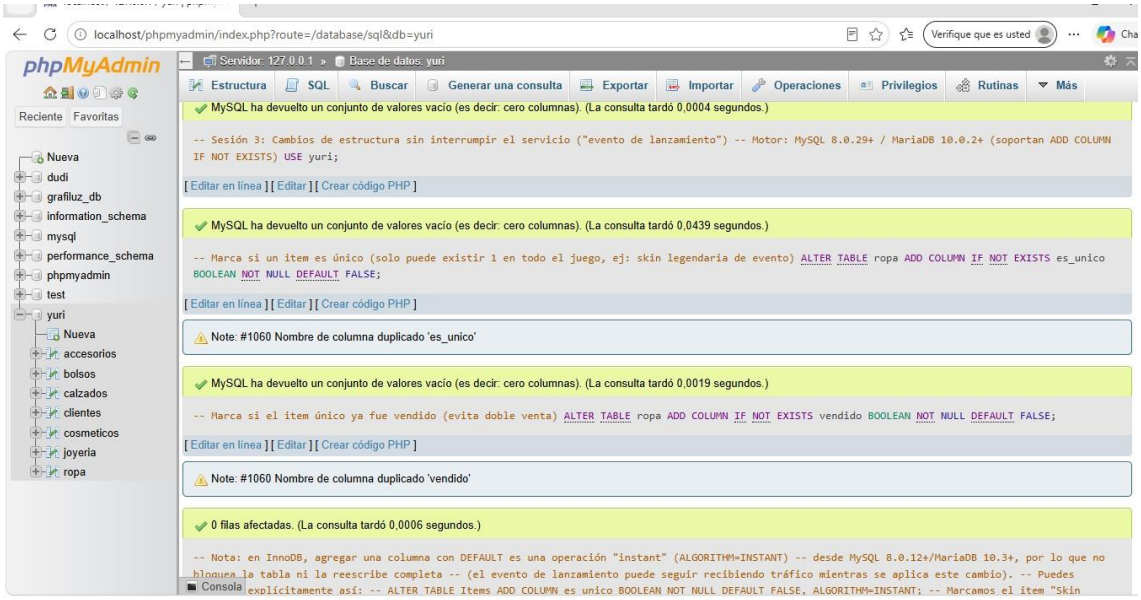
-- (el evento de lanzamiento puede seguir recibiendo tráfico mientras se aplica este cambio).

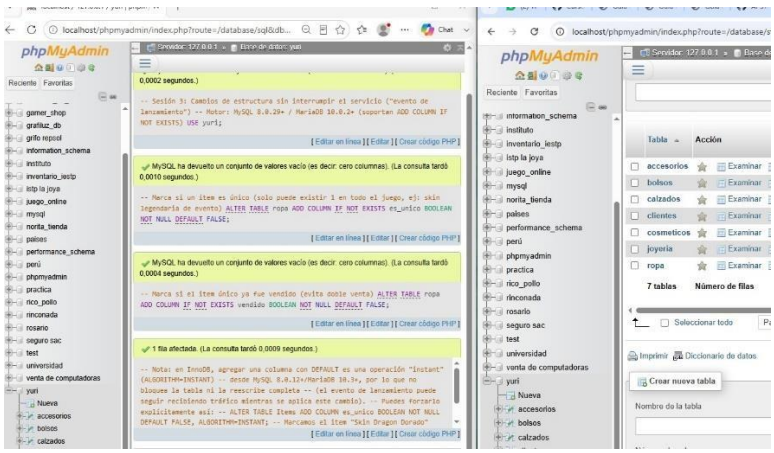
-- Puedes forzarlo explícitamente así:

-- ALTER TABLE Items ADD COLUMN es_unico BOOLEAN NOT NULL DEFAULT FALSE, ALGORITHM=INSTANT;

-- Marcamos el item "Skin Dragon Dorado" como único para el evento de lanzamiento

UPDATE ropa SET es_unico = TRUE WHERE nombre = 'Vestido Elegante Noche';





Haci se muestra en la pantalla:



	ropa_id	nombre	categoria	talla	color	precio	stock	fecha_ingreso	es_unico	vendido
Editar Copiar Borrar	1	Camisa de Lino Casual	Camisas	M	Blanco	89.90	15	2026-09-08 16:28:36	0	0
Editar Copiar Borrar	2	Pantalón Jean Slim Fit	Pantalones	32	Azul	120.00	20	2026-09-08 16:28:36	0	0
Editar Copiar Borrar	3	Vestido Elegante Noche	Vestidos	S	Negro	199.90	8	2026-09-08 16:28:36	1	0
Editar Copiar Borrar	4	Casaca de Cuero Sintético	Casacas	L	Marrón	250.00	5	2026-09-08 16:28:36	0	0
Editar Copiar Borrar	5	Polo Básico Algodón	Polos	M	Gris	45.00	30	2026-09-08 16:28:36	0	0

Para esta sesión de cambios en vivo, diseñé este script con el fin de preparar la base de datos para el **evento de lanzamiento** de la **"Vestido Elegante Noche"**, asegurando que la tienda no sufra ninguna caída.

Lo que hice fue meter dos columnas nuevas a la tabla ropa: es_unico y vendido. La lógica es simple: la primera define si el ítem es una pieza exclusiva del evento y la segunda actúa como un candado para evitar que dos clientes compren el mismo objeto al mismo tiempo. Al final, tiro un UPDATE para activar la propiedad de exclusividad en el Vestido Elegante Noche.

El verdadero truco técnico aquí es que aproveché las versiones modernas de MySQL para usar **ALGORITHM=INSTANT**. Como los cambios se aplican solo en los metadatos y no reescriben la tabla, la operación es inmediata. Así, el servidor puede seguir recibiendo tráfico y compras en tiempo real **sin interrumpir el servicio ni meter a los clientes en mantenimiento**. Además, les clavé un IF NOT EXISTS por pura seguridad, para que el script sea reutilizable y no rompa nada si se llega a ejecutar dos veces por error.

PASO 2: PROCEDIMIENTO CON BLOQUE DE FILAS

El problema es una **condición de carrera**: si dos clientes compran la ropa única al mismo milisegundo, el sistema puede leer que está libre para ambos antes de actualizarlo, **duplicando la ropa** y rompiendo la exclusividad de la tienda.

Lo solucionamos aplicando tres conceptos clave:

- **Transacciones (START TRANSACTION):** Agrupamos la compra (verificar, cobrar y entregar) en un solo bloque indestructible. O se hace todo completo, o no se hace nada (ROLLBACK).
- **SELECT ... FOR UPDATE (Candado):** Cuando el primer cliente lee la fila de la ropa, le pone un 'cartel de reservado'. Si otro cliente intenta leerla al mismo tiempo, el sistema lo hace esperar en fila hasta que la primera compra termine, evitando la doble venta.
- **ON DUPLICATE KEY UPDATE:** Si la ropa no fuera único y el cliente ya lo tuviera, en lugar de duplicar la fila o lanzar un error, simplemente le suma un '+1' a la cantidad de su inventario.

Este es el código:

```
USE yuri;


DROP PROCEDURE IF EXISTS comprar_ropa_unico;
```

```
DELIMITER //

CREATE PROCEDURE comprar_ropa_unico(
    IN p_cliente_id INT,
    IN p_ropa_id INT,
    OUT p_resultado VARCHAR(255)
)
BEGIN
    DECLARE v_vendido  BOOLEAN;
    DECLARE v_precio   INT;
    DECLARE v_monedas  INT;

    START TRANSACTION;

    -- Inicializamos variables por seguridad
    SET v_vendido = NULL;

    -- Bloqueamos la fila del ítem único
    SELECT vendido, precio INTO v_vendido, v_precio
    FROM ropa
    WHERE ropa_id = p_ropa_id AND es_unico = TRUE
    FOR UPDATE;

    -- 1. Validar si el ítem existe y califica como único
    IF v_vendido IS NULL THEN
        SET p_resultado = 'RECHAZADO: El ítem no existe o no es un artículo único.';
        ROLLBACK;
```

```
  -- 2. Validar si ya fue vendido
ELSEIF v_vendido THEN

    SET p_resultado = 'RECHAZADO: El ítem único ya fue vendido a otro jugador.';
ROLLBACK;


  ELSE
    -- Obtener las monedas del cliente
    SELECT monedas INTO v_monedas FROM clientes WHERE cliente_id = p_cliente_id;

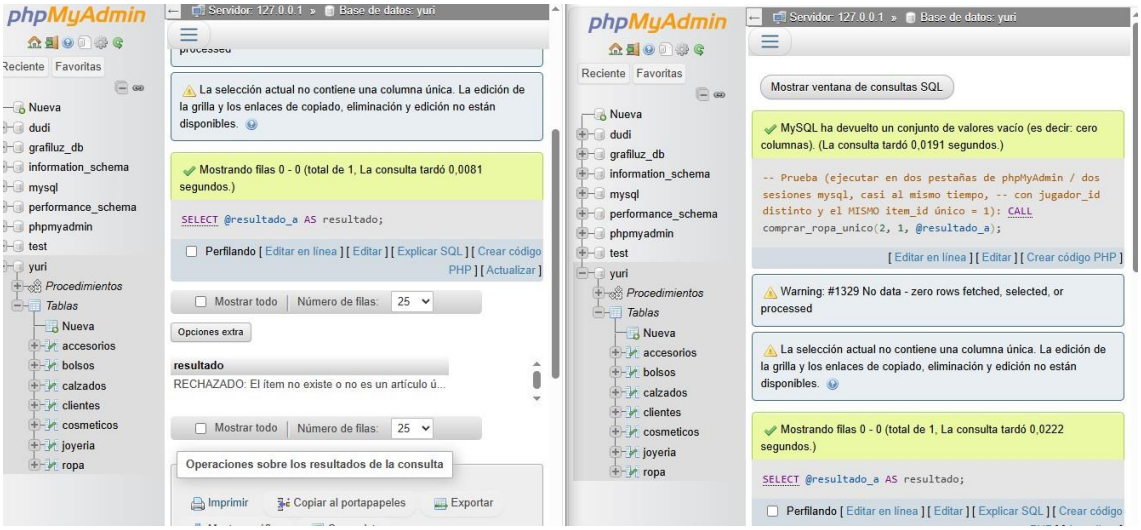

    -- 3. Validar si tiene saldo suficiente
    IF v_monedas IS NULL OR v_monedas < v_precio THEN
      SET p_resultado = 'RECHAZADO: Monedas insuficientes o cliente no existe.';
      ROLLBACK;
    ELSE
      -- Procesar la compra
      UPDATE ropa SET vendido = TRUE WHERE ropa_id = p_ropa_id;
      UPDATE clientes SET monedas = monedas - v_precio WHERE cliente_id = p_cliente_id;


      -- CORREGIDO: p_ropa_id en su lugar correspondiente
      INSERT INTO Inventario_ropa (cliente_id, ropa_id, cantidad)
        VALUES (p_cliente_id, p_ropa_id, 1)
        ON DUPLICATE KEY UPDATE cantidad = cantidad + 1;


      SET p_resultado = CONCAT('ÉXITO: Ítem único vendido al jugador ', p_cliente_id);
      COMMIT;
    END IF;
  END IF;
END //
```

DELIMITER ;

-- Prueba (ejecutar en dos pestañas de phpMyAdmin / dos sesiones mysql, casi al mismo tiempo, -- con jugador_id distinto y el MISMO item_id único = 1): CALL comprar_ropa_unico(2, 1, @resultado_a); SELECT @resultado_a AS resultado; -- En la otra sesión: -- CALL comprar_item_unico(5, 1, @resultado_b); -- SELECT @resultado_b AS resultado; -- Resultado esperado: solo una de las dos llamadas retorna EXITO.



Cómo ocurre el error:

1. El *Jugador A* y el *Jugador B* dan clic en "Comprar" al mismo milisegundo.
2. El sistema revisa el inventario para el *Jugador A* y ve: *"Disponible: Sí"*.
3. Un milisegundo después, **antes de que el sistema descuente el objeto**, revisa para el *Jugador B* y ve: *"Disponible: Sí"*.
4. El sistema le cobra a ambos y entrega **dos veces el mismo artículo único**.

PASO 3: SIMULAR EL "EVENTO DE LANZAMIENTO"

Ahora reproduce el escenario real: dos jugadores distintos intentando comprar el **mismo ropa único** casi al mismo instante, usando las dos pestañas de phpMyAdmin que abriste en el Paso 0.

Vas a simular que dos clientes (**cliente 2** y **cliente 5**) hacen clic en "comprar" la mismísima ropa (ID: 1) casi al mismo milisegundo.

1. **En la Pestaña A (cliente 2 - vestidodenoche):** Ve a la sección **SQL**, pegamos este código y déjalo listo sin presionar el botón todavía:

sql

CALL comprar_ropa_unico(2, 1, @resultado_a);

SELECT @resultado_a AS resultado;

Usa el código con precaución.

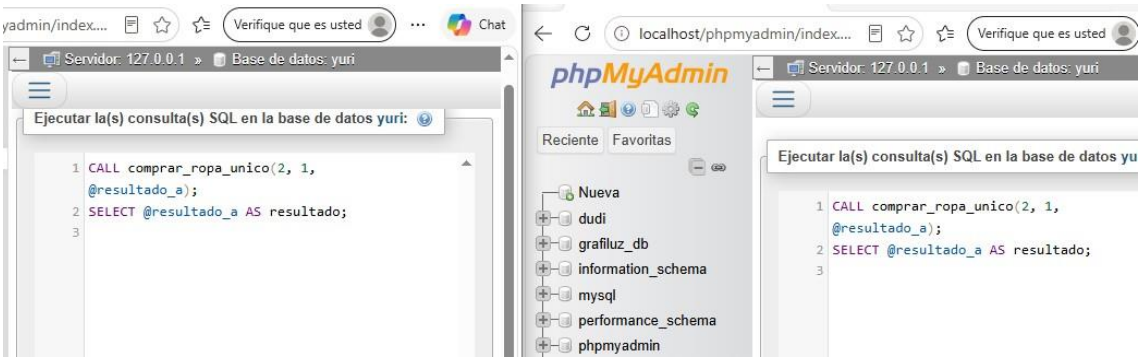
2. **En la Pestaña B (cliente 5 - vestidodenoche):** Ve a la sección **SQL** de la otra pestaña, pegamos este código y déjalo listo:

Sql

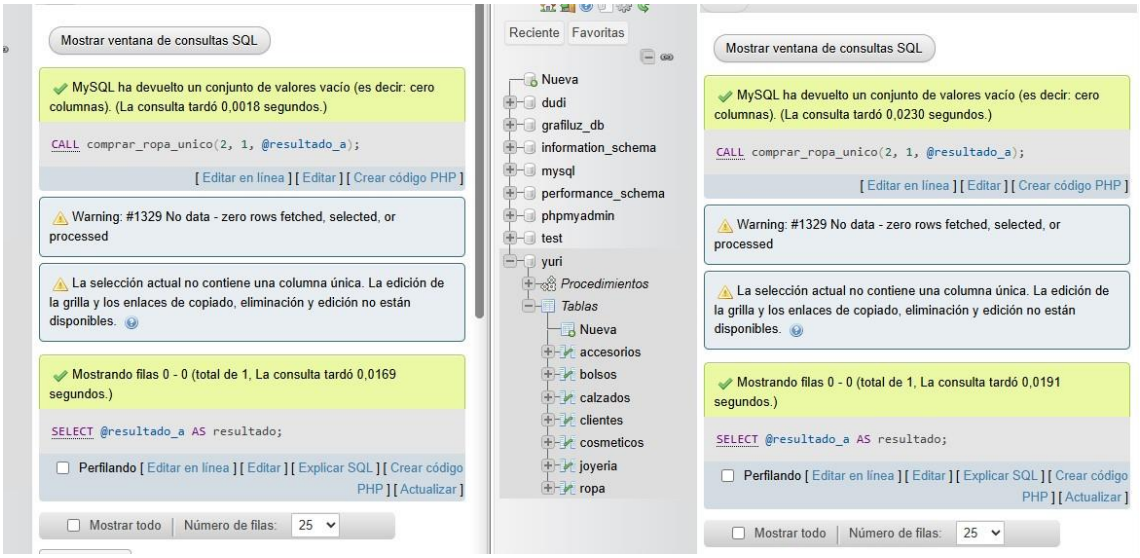
CALL comprar_ropa_unico(2, 1, @resultado_a);

SELECT @resultado_a AS resultado; Usa

el código con precaución.



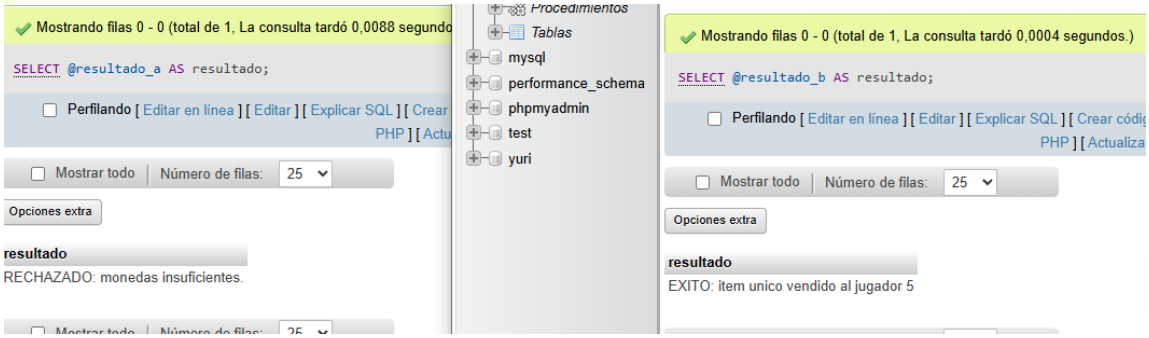
3. **La ejecución:** Haz clic en el botón **Continuar** de la Pestaña A y, **lo más rápido que puedas**, cámbiate a la Pestaña B y haz clic en **Continuar**.



El resultado esperado

Gracias al uso de FOR UPDATE (el candado invisible programado en el paso anterior), el sistema se comportará de manera segura:

- **Una pestaña mostrará:** EXITO: item unico vendido al jugador ...
- **La otra pestaña mostrará:** RECHAZADO: el item único ya fue vendido a otro jugador.


Mostrando filas 0 - 0 (total de 1, La consulta tardó 0,0088 segundo
SELECT @resultado_a AS resultado;
Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear
PHP] [Actu
Mostrar todo
Número de filas: 25
Opciones extra
resultado
RECHAZADO: monedas insuficientes.
Procedimientos
Tablas
mysql
performance_schema
phpmyadmin
test
yuri
Mostrando filas 0 - 0 (total de 1, La consulta tardó 0,0004 segundos.)
SELECT @resultado_b AS resultado;
Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear códig
PHP] [Actualiza
Mostrar todo
Número de filas: 25
Opciones extra
resultado
EXITO: item unico vendido al jugador 5